

Bacharelado em Ciência da Computação
Disciplina: PAA
Professor: Mário Luiz Rodrigues Oliveira

Teoria da computabilidade (Decidibilidade)

Decidíveis

Indecidíveis

Teoria da complexidade

Intratáveis: não são conhecidas soluções polinomiais determinísticas e nem se sabe se elas existem. (supercomputadores???, computadores quânticos ??? Ex: Caixeiro Viajante (TSP), para 26 cidades e considerando 4 bilhões de instruções por segundo; solução daqui $123x10^6$ anos !!!)

Tratáveis: existem soluções polinomiais determinísticas

Análise de algoritmos trata de: provar que um algoritmo está correto e estimar/prever o consumo de recursos pelo algoritmo - Corretude e eficiência

Critérios: tempo, espaço, simplicidade e facilidade de codificação, boas práticas de engenharia de software ...

Análise empírica x modelo matemático (analítico)

Análise empírica: implementam-se os algoritmos e realizam-se testes para escolher o melhor. Problemas com essa abordagem:

- **necessidade de implementar o algoritmo;**
- **impossibilidade de testar todas as entradas possíveis;**
- **dificuldade na definição do espaço amostral;**
- **habilidade do programador influencia no resultado;**

- **a linguagem de programação adotada, o compilador; a quantidade de memória, hierarquia de memória, o sistema operacional e arquitetura do sistema computacional interferem no resultado**

Existem situações nas quais a análise empírica é indicada???

Modelo matemático: prever os recursos que um algoritmo necessitará. O objetivo é modelar o comportamento do algoritmo sem implementá-lo. Comparam-se algoritmos e não programas. Problemas com essa abordagem: definição das variáveis usadas no modelo matemática e análise do comportamento do algoritmo para possíveis cenários

Há a necessidade de definir um modelo de computação genérico/idealizado para efetuarmos a análise teórica da complexidade de algoritmos.

Neste curso adota-se o modelo RAM. Os custos das operações no modelo RAM são:

- **o tempo de um comando de atribuição, leitura ou escrita é O(1)**
- **o tempo para avaliar a(s) condição(ões) de um comando de decisão é O(1)**
- **o tempo para avaliar a condição de término de um comando de repetição é O(1)**

Não há um conjunto completo de regras para analisar um algoritmo. No entanto, existem alguns princípios a serem seguidos, tais como:

- **o tempo de execução de uma sequencia de comandos é determinado pelo maior tempo de execução de qualquer comando da sequencia**
- **o tempo de execução de um comando de decisão/seleção é composto pelo tempo de execução dos comandos executados dentro do comando de decisão/seleção, acrescido do tempo de avaliar a condição. O tempo de avaliar a condição, como visto, é O(1)**

- **o tempo para executar um comando de repetição (laço) é a soma do tempo de execução do corpo laço mais o tempo de avaliar a condição de término, multiplicado pelo número de iterações do laço. E o tempo para avaliar a condição de término de um comando de repetição é O(1)**

Devemos escolher uma operação básica/elementar/relevante/mais significativa. Qual operação escolher? A operação que mais contribui para o tempo de execução do algoritmo

A análise da eficiência de um algoritmo concentra na ordem de crescimento da operação básica. Assim devemos contar quantas vezes as operações básicas são executadas

É usual definir uma função de complexidade para medir o tempo de execução de um algoritmo

Uma função de complexidade de tempo f(n) mede o tempo necessário para executar um algoritmo em um problema de tamanho n

A complexidade de tempo na realidade não representa tempo diretamente, mas o número de vezes que determinada operação considerada relevante é executada

Assim, a análise da eficiência de algoritmos se concentra na ordem de crescimento da operação básica de um algoritmo, como o principal indicador de sua eficiência

Funções de complexidade têm como parâmetro o tamanho da entrada e para alguns algoritmos a eficiência depende, também, do tipo da entrada e não apenas do tamanho da entrada: pior caso, melhor caso e caso médio

Preferência para o pior caso: maior tempo de execução sobre todas as entradas de tamanho n. Fornece um limite superior para o tempo de execução do algoritmo. É de fácil identificação e cálculo. Para diversos algoritmos ocorre com frequência e em muitos casos há uma proximidade entre o pior caso e o caso médio

Melhor caso: menor tempo de execução sobre todas as entradas de tamanho n. É de pouca utilidade, pois pode nunca ocorrer na prática. Não é uma boa escolha

Caso médio: É a média dos tempos de execução de todas as entradas de tamanho n. Na teoria é necessário supor distribuições de probabilidade. É o caso esperado e intuitivamente seria o caso ideal. Pode envolver problemas matemáticos complexos

O tamanho n da entrada fornece uma medida de dificuldade de resolver o problema

Para valores pequenos de n qualquer algoritmo custa pouco

A análise de algoritmos é realizada para grandes valores de n

Usa o conceito de comportamento assintótico e notações Big O, Big Ômega e Teta.

Notação Big O indica uma relação de pertinência. Assim, f(n) = O (g(n)), g(n) é o limite de f(n) e f(n) cresce numa taxa não maior que g(n). Embora a literatura tenha consagrado o uso do sinal de igualdade, é preferível dizer f(n) pertence a O(g(n)) e não f(n) = O (g(n))

Formalmente, f(n) <= c * g(n), para uma constante real positivo c e para todo n >= m. As constantes n e m são valores inteiros positivos.

Essa definição estabelece que somente precisam existir certos c e m, mas não dá nenhuma sugestão de como quaisquer esses valores constantes. No geral, existem infinitos pares de c e m que garantem a relação f(n) = O (g(n))

Desprezam-se constantes aditivas e multiplicativas e termos de menor ordem em taxa de crescimento

A análise assintótica reduz o problema a uma resposta menos precisa, porém é fácil de derivar e interpretar. Não há necessidade de preocupar com valor da constante envolvida

As constantes dependem do tempo real e exato de cada operação. O modelo de computador

genérico adotado (RAM) ignora o custo real das operações. Dessa forma, ignoramos as constantes

Não se diz T(n) = O(2n) e sim T(n) = O(n)

Mudar o hardware afeta a taxa de crescimento por um fator constante, mas não altera a ordem de complexidade dos algoritmos

A notação Ômega é definida formalmente como f(n) >= c * g(n), para uma constante real positivo c e para todo n >= m. As constantes n e m são valores inteiros positivos.

A notação Teta aplica-se à uma função que é simultaneamente Big O e Big Ômega.

Técnicas de análise de algoritmos são consideradas partes integrantes do processo moderno de resolver problemas, permitindo

escolher, de forma racional, um dentre vários algoritmos disponíveis para uma mesma aplicação. Se a mesma medida de custo é aplicada a diferentes algoritmos, então é possível compará-los e escolher o mais adequado para resolver o problema em questão

Assim, ao compararmos 2 ou mais algoritmos podemos considerar como mais eficiente o com menor taxa de crescimento. Lembre-se essa afirmativa é válida desde que o tamanho da entrada seja suficientemente grande (comportamento assintótico)

A complexidade de um problema é a complexidade do melhor algoritmo que o resolve

Tendo feito a análise assintótica, como verificar que a implementação do algoritmo tem o desempenho previsto pela análise?

O que fazer quando os resultados de sua análise não estão de acordo com o comportamento observado do programa?

Como comparar os resultados se a análise assintótica considera valores quando n tende ao infinito e o tempo real de processamento deve ser medido para valores finitos e determinados de n?

Pode-se calcular a taxa entre T(n) (tempo medido) e f(n) (função de complexidade) para cada valor de n do experimento e observar como a taxa se comporta à medida que n cresce. Se a taxa convergir para uma constante, então, provavelmente, a análise está correta.

Se a taxa divergir:

- **f(n) não é um limite restrito**
- **f(n) está subdimensionada**

- **foi feita uma análise para o pior caso e o mesmo não foi contemplado nos testes experimentais**
- **eventos do sistema computacional (escalonamento processos, memória cache, memória virtual e outros) estão influenciando nas tomadas de tempo real**
- **houve um erro e a análise está incorreta**

Referências Bibliográficas:

ZIVIANI, Nivio. Projeto de Algoritmos – Com implementação em Pascal e C. 3ª edição revista e ampliada. São Paulo: Cengage Learning, 2011

CORMEN, T. et al. Algoritmos: Teoria e Prática. Editora Campus, 2002

PREISS, Bruno R. Estruturas de Dados e Algoritmos – padrões de projetos orientados a objetos com Java. Editora Campus, 2001

DROZDEK, Adam. Estruturas de dados e algoritmos em C++. Cengage Learning, 2002

SIPSER, Michael. Introdução à Teoria da Computação. 2ª edição. Cengage Learning, 2007